[Dinu Barbu]

What happens when an agent reads your prompt.

Six sections, one idea: an agent is a program passing text to a text predictor, over and over. Everything that feels mysterious about it — **memory, tools, context limits, cost** — falls out of that one arrangement.

// an explainer

↳ dvinubius · dinubarbu.com

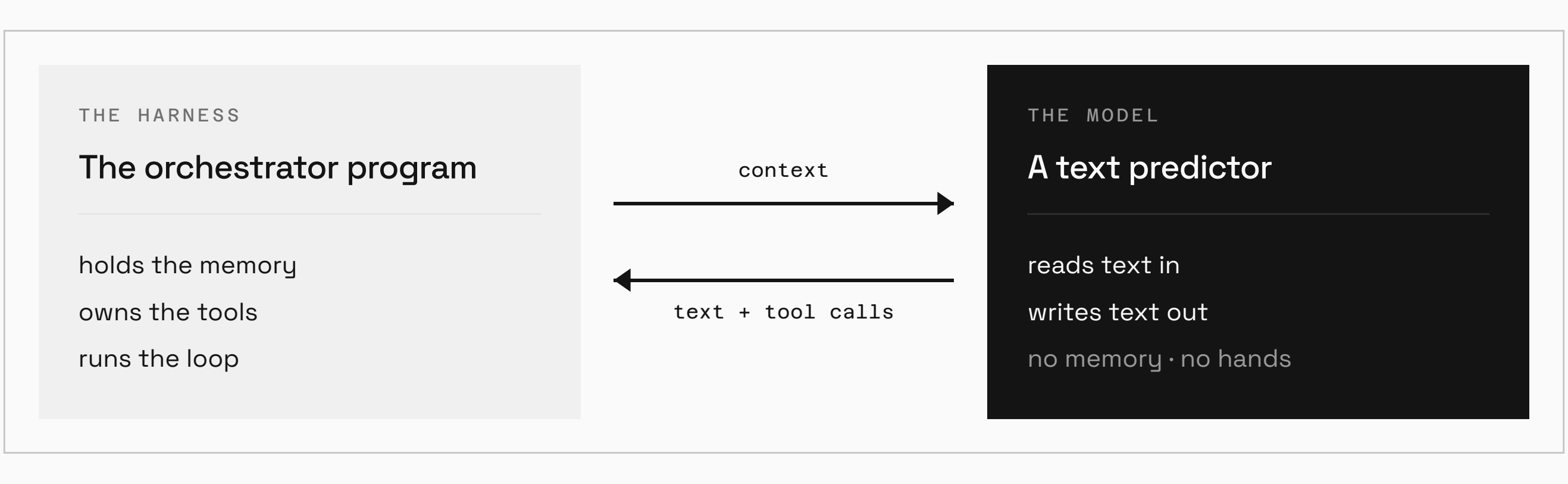
---

01 / the mental model · keep this one

What an agent actually is

An agent is a conversation between a **harness** — the orchestrator program — and a **model**, which is a text predictor. The harness hands over a big blob of text; the model hands back text. That exchange is the whole game.

fig. 1 · the two participants, and the only two things that cross between them



ink is the model, shade is the harness — and that holds for every figure below

Everything else here is detail on those two arrows: what is **in** the context the harness sends, and what the harness **does** with the text it gets back.

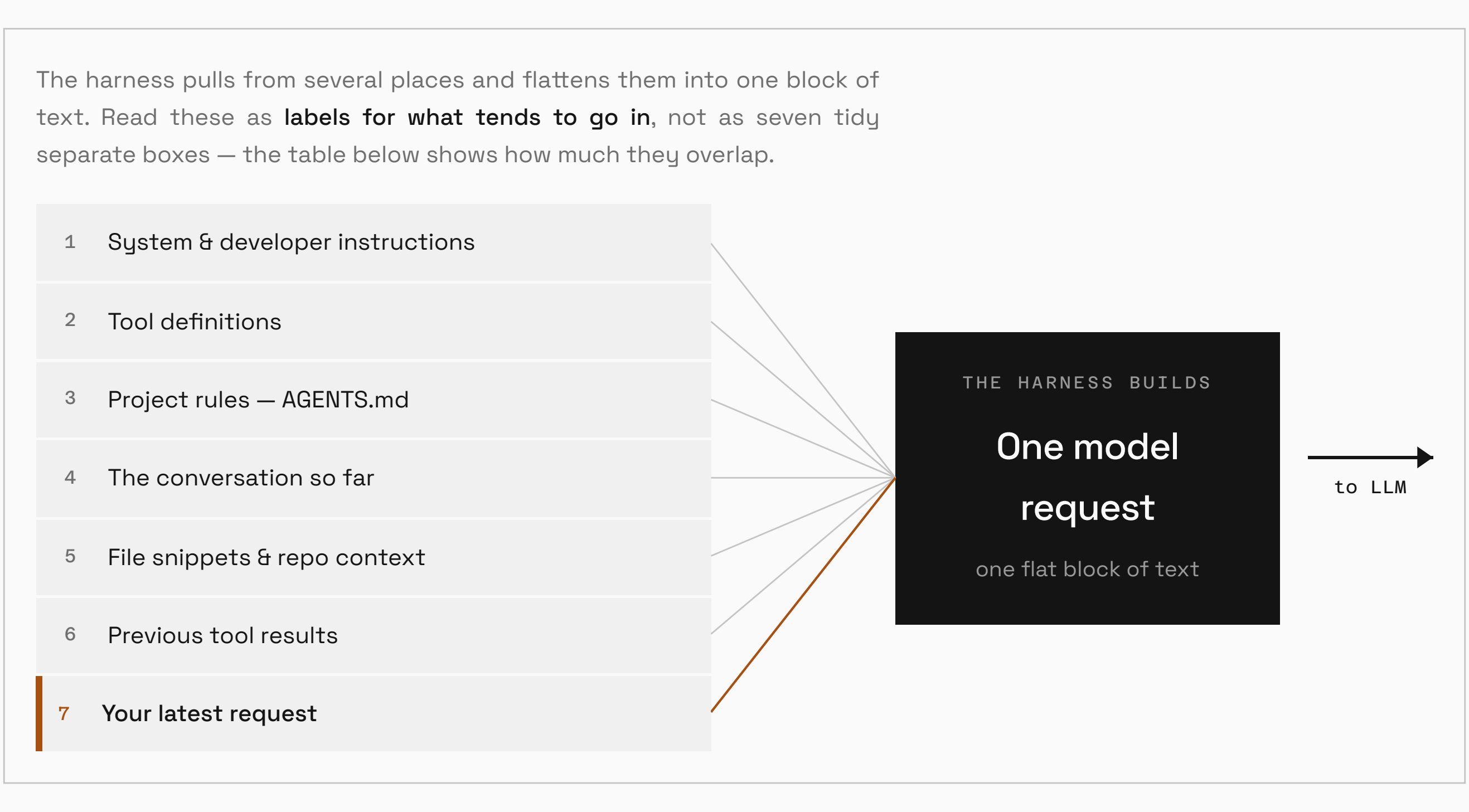
---

02 / the first arrow · what the model is handed

Context is everything the model sees

The model has no window into your machine. Before every call the harness assembles one request — a single stack of text. If something is not in this stack, the model does not know it exists.

fig. 2 · building the request


The harness pulls from several places and flattens them into one block of text. Read these as **labels for what tends to go in**, not as seven tidy separate boxes — the table below shows how much they overlap.

the marked row is the only one you author — everything above it was put there by the harness or by an earlier turn

Here is what each label points at, and where the seven stop being seven.

fig. 3 · the seven labels, and the line that runs through them

one model request — assembled fresh each turn

1. **System & developer instructions**
   the model's standing rules — how to behave, when to stop
2. **Tool definitions**
   the menu of actions it can request — read, search, run, edit
3. **Project rules** `AGENTS.md`
   your repo's conventions, do's and don'ts

↓ ITEMS 4–7 ARE REALLY ONE THING: THE CONVERSATION

4. **The conversation so far**
   the whole back-and-forth this session — and it already **contains** 5 and 6
5. **File snippets & repo context** `part of #4`
   code the model pulled — it arrived earlier as a tool result
6. **Previous tool results** `part of #4`
   outputs, errors and diffs from earlier tool runs
7. **Your latest message** `newest line of #4`
   the new thing you just asked for

the band is the real boundary in this table — above it, three separate things; below it, one growing list

> **These labels overlap. They are a simplification, not seven disjoint boxes.**
>
> Only 1–3 are truly separate. Everything else is one conversation: #4 is the whole back-and-forth, which already holds the file snippets (#5) and the tool results (#6) the model pulled, with your latest message (#7) as its newest line. They are split out only to name what tends to go in. Section 05 shows the real, non-overlapping layout — stable prefix on top, one ordered conversation below.

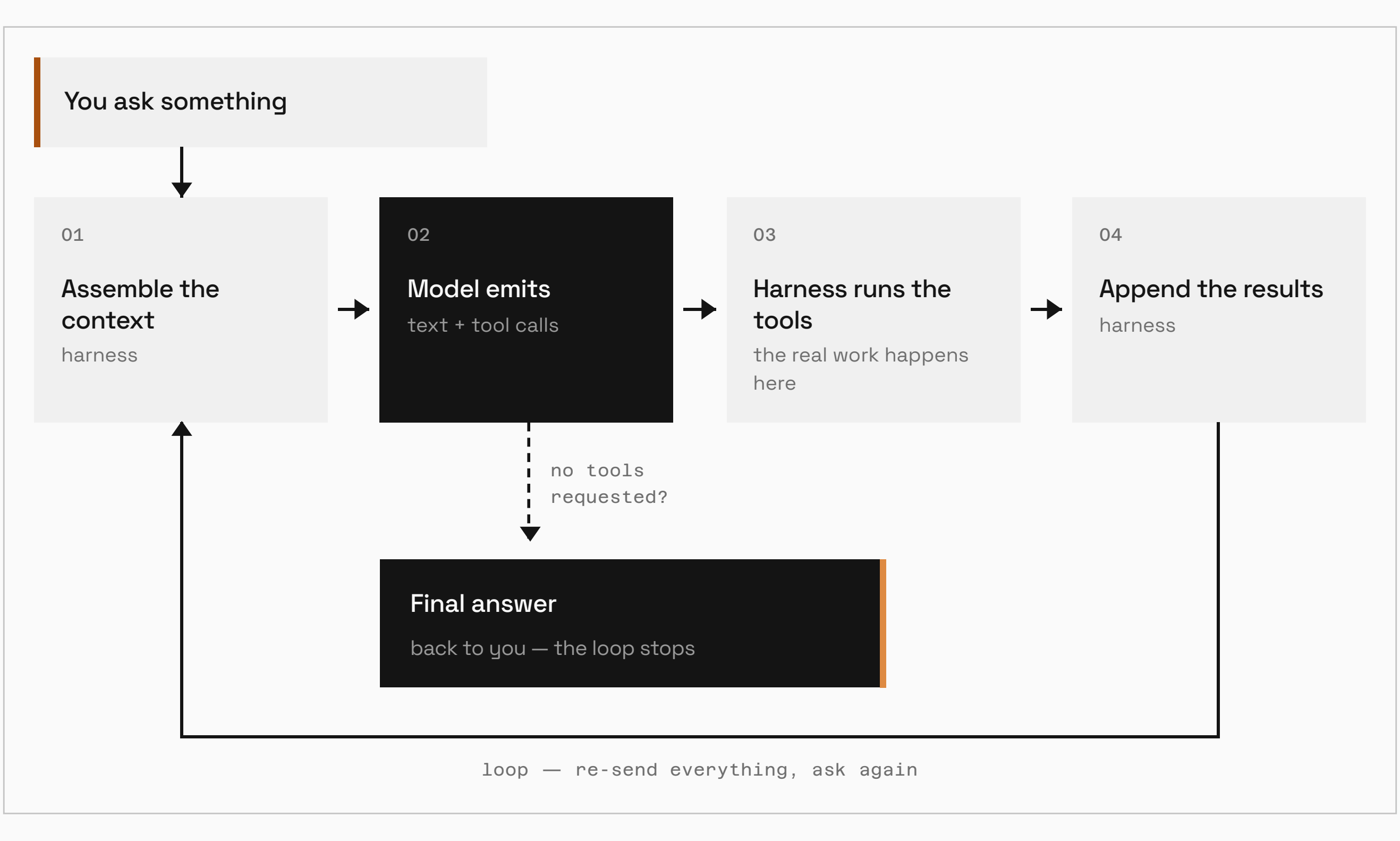
---

03 / the signature move · the loop

One prompt becomes many turns

The reply usually is not an answer yet — it is a request to **do** something. So the harness acts, folds the result back into the context, and asks again. Around and around, until the model has nothing left to ask for.

fig. 4 · the loop, and the two points you touch it



the two marked rows are where a human touches this — you open the loop and you receive what closes it; the four steps between them run without you

Steps 1 and 4 are bookkeeping
The harness builds the text going in, and stitches results back in after.

Step 2 is the only model call
It thinks once, then yields control. It cannot act on its own.

Step 3 is the real work
Files get read, tests get run, edits get applied — all by the harness.

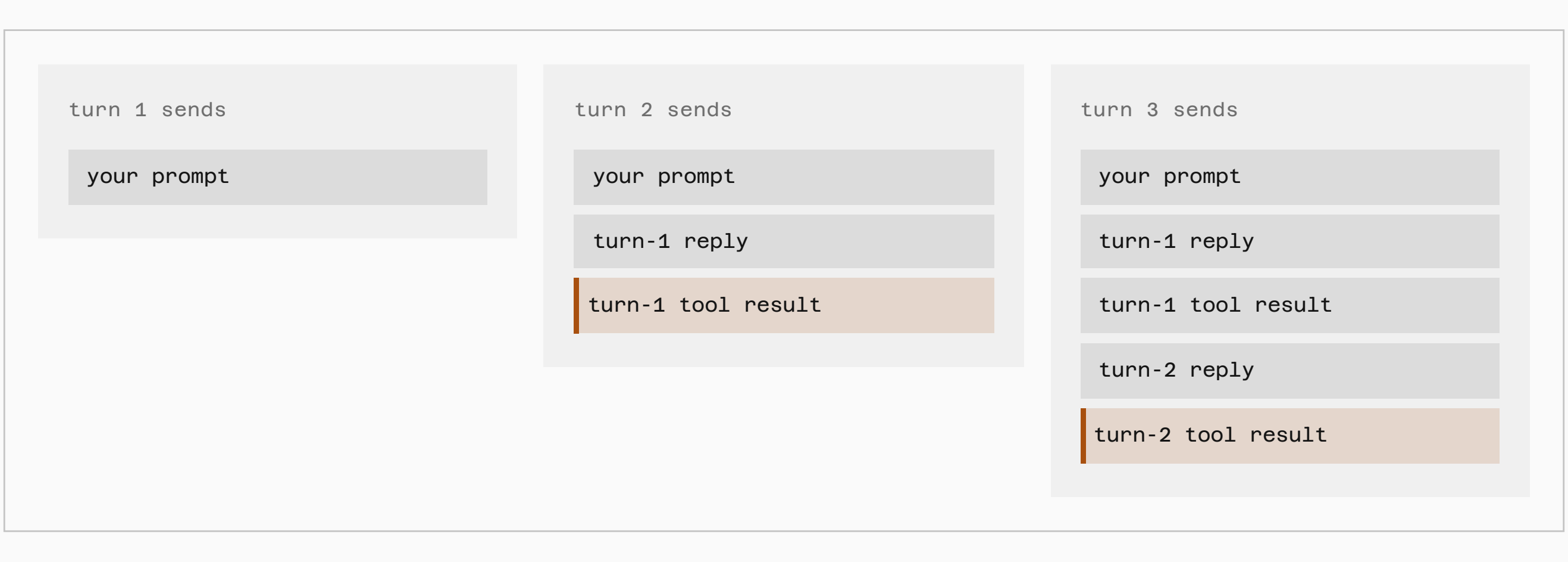
---

04 / the detail that makes it click

Memory is the harness re-feeding the whole transcript

Because the model forgets everything between calls, the harness cannot just send the new result. It re-sends the **entire growing history** every single turn. Each call is bigger than the last.

fig. 5 · what three consecutive turns actually send



the marked bar in each turn is the only genuinely new text in that request — the tool result the harness had just handed back

> This is why agents have a **context limit**, and why long sessions slow down or start forgetting early details. The transcript eventually gets too big to re-send in full, so the harness has to trim it or summarize it.

---

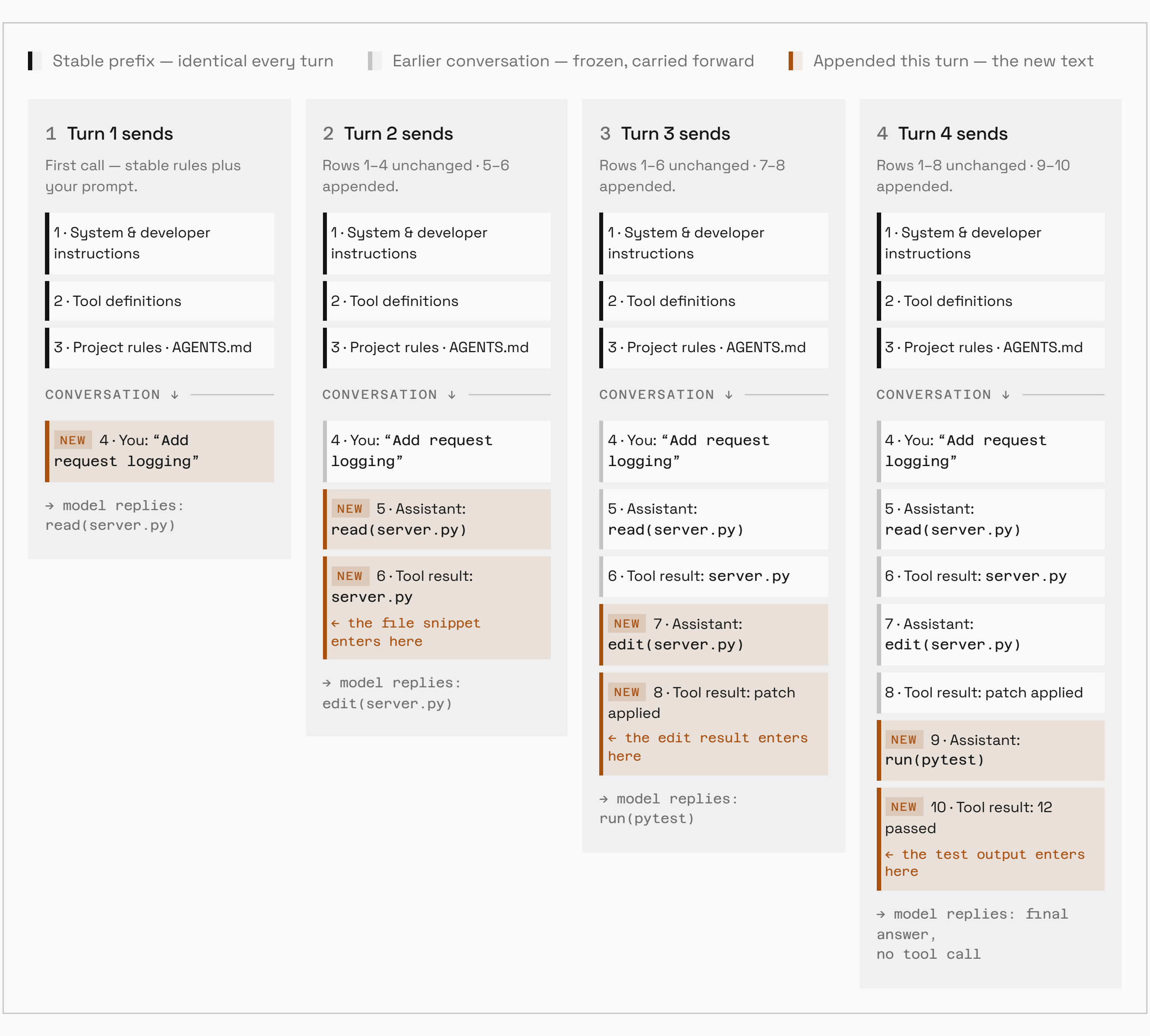
05 / look closer · one request, fully expanded

What each turn actually sends

The seven sources are not seven floating slots. The harness lays them out as **one ordered sequence**: the stable three at the top, then the conversation — and the conversation absorbs file snippets and tool results as it grows.

Below is one full cycle of the request `Add request logging to the server` — four model calls, then a final answer. Watch it grow by appending only. Earlier rows never change.

fig. 6 · four turns of one cycle, row by row


Stable prefix — identical every turn
Earlier conversation — frozen, carried forward
Appended this turn — the new text
1 Turn 1 sends
First call — stable rules plus your prompt.
1 · System & developer instructions
2 · Tool definitions
3 · Project rules · AGENTS.md
CONVERSATION ↓
NEW 4 · You: "Add request logging"
→ model replies: read(server.py)
2 Turn 2 sends
Rows 1–4 unchanged · 5–6 appended.
1 · System & developer instructions
2 · Tool definitions
3 · Project rules · AGENTS.md
CONVERSATION ↓
4 · You: "Add request logging"
NEW 5 · Assistant: read(server.py)
NEW 6 · Tool result: server.py
← the file snippet enters here
→ model replies: edit(server.py)
3 Turn 3 sends
Rows 1–6 unchanged · 7–8 appended.
1 · System & developer instructions
2 · Tool definitions
3 · Project rules · AGENTS.md
CONVERSATION ↓
4 · You: "Add request logging"
5 · Assistant: read(server.py)
6 · Tool result: server.py
NEW 7 · Assistant: edit(server.py)
NEW 8 · Tool result: patch applied
← the edit result enters here
→ model replies: run(pytest)
4 Turn 4 sends
Rows 1–8 unchanged · 9–10 appended.
1 · System & developer instructions
2 · Tool definitions
3 · Project rules · AGENTS.md
CONVERSATION ↓
4 · You: "Add request logging"
5 · Assistant: read(server.py)
6 · Tool result: server.py
7 · Assistant: edit(server.py)
8 · Tool result: patch applied
NEW 9 · Assistant: run(pytest)
NEW 10 · Tool result: 12 passed
← the test output enters here
→ model replies: final answer,
no tool call

every row that has been written once keeps its exact position and its exact bytes — the request only ever grows at the bottom

↳ Turn 4's reply is what closes the cycle

```
11 · Assistant (final): "Added logging middleware — 12 tests pass."

// no tool call → the harness stops looping
// and hands this back to you
```

Row 11 is the model's reply to turn 4. It is not **sent** in another request; it is the output that ends the loop. Whatever you type next appends as **row 12**, and the harness fires turn 5 on top of all eleven rows — the same cycle, continued.

> **So the conversation so far is everything from row 4 down — and it is not only your messages.**
>
> The model's own replies, the files it read, and every tool result are all entries in that one growing list. File snippets and previous tool results are not separate slots beside the conversation — they **enter** the conversation as tool-result messages, in order, and then never change.

One simplification: this example does a single tool call per turn — read, then edit, then test — to keep the steps legible. A real agent often batches several calls into one reply, which collapses this into fewer turns. The append-only growth is identical either way; only the number of turns changes.

---

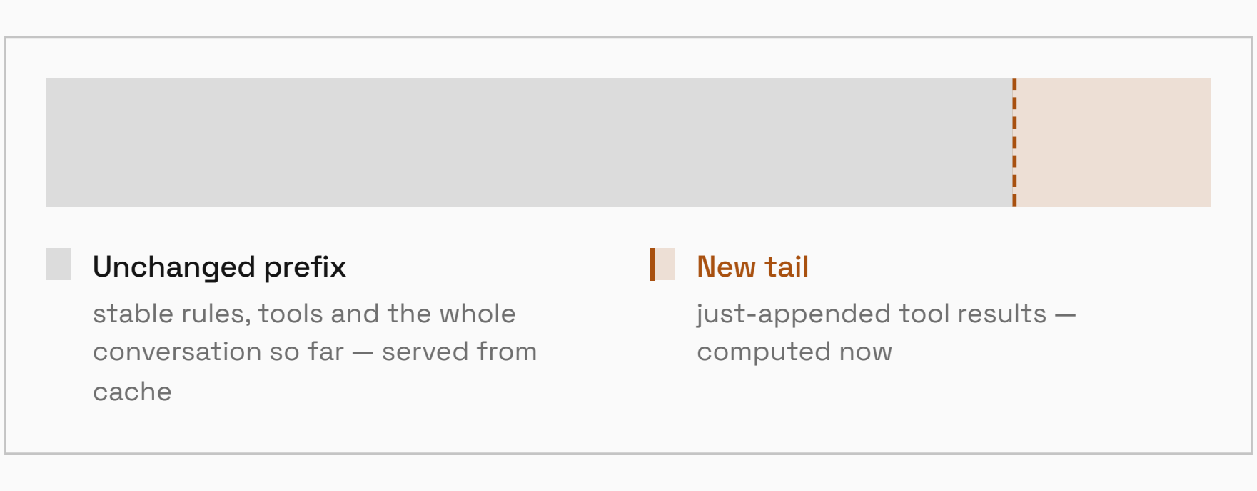
06 / the payoff · why append-only matters

Why the prefix has to stay byte-for-byte identical

A model normally re-reads its entire input from scratch on every call. The **KV cache** is how it avoids that: for every token it has already processed it stores the intermediate attention values — the **keys** and **values**. Hand it the exact same opening sequence again and it skips straight past those tokens, computing only what is new at the end.

That is the real reason the harness re-sends everything unchanged and only appends. Each turn, the long stable prefix **and** the entire prior conversation are a cache hit. Because the model already computed its own replies while generating them, the only genuinely new text it has to read each turn is the **tool results** the harness just handed back.

fig. 7 · one request, left to right


Unchanged prefix
stable rules, tools and the whole conversation so far — served from cache
New tail
just-appended tool results — computed now

the dashed edge is where cached work ends and fresh computation begins — everything to the left of it was paid for on an earlier turn

> **Change anything early and every cached value after the change is invalid.**
>
> Edit AGENTS.md or the system prompt, reorder messages, or trim and summarize old turns to fit the context limit — the model has to recompute from that point on. **Append-only is cheap; rewriting history is expensive.** It is why editing AGENTS.md mid-session costs more than it looks: it sits near the top, so the whole conversation below it falls out of cache. And it is why trimming a long conversation — the fix section 04 ends on — is not free either.

[Dinu Barbu] · Software Engineer — Agentic Developer Workflows
↳ dvinubius · dinubarbu.com